FPGA Tick-to-Trade

Part 1 — Foundation & Architecture

Abhishek · June 2026 · AWS F1 (UltraScale+ VU9P)

Target roles: FPGA Engineer / HFT Infrastructure — Citadel, DRW, Jump Trading, Optiver

Contents

1 Project Goal and Motivation

The goal is to build a **portfolio-grade FPGA trading system** that demonstrates the exact engineering skills HFT firms hire for:

- RTL design in SystemVerilog — not HLS, not C/C++ with Vivado HLS
- Binary protocol parsing in hardware (NASDAQ TotalView-ITCH 5.0)
- Pipelining, fixed-point arithmetic, and hardware latency measurement
- Rigorous verification: golden-model comparison, cocotb, QuestaSim

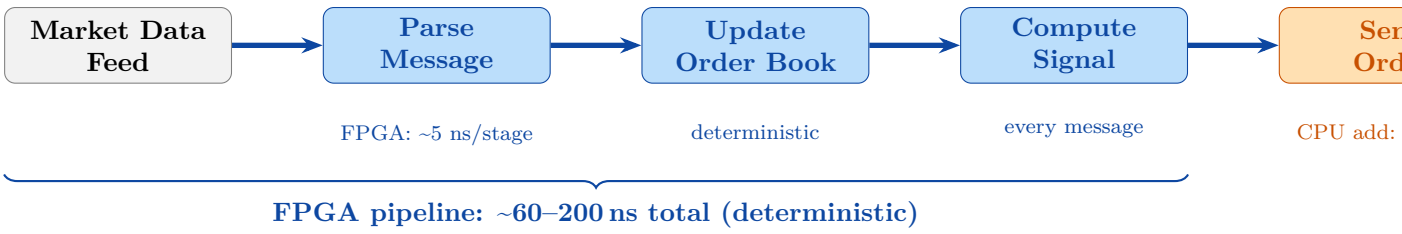
The project is targeted at **FPGA Engineer** and **HFT Infrastructure** roles at firms like Citadel Securities, DRW, Jump Trading, and Optiver, in the \$125k–\$350k compensation range.

2 Background: Why Latency Matters in HFT

2.1 The Speed Arms Race

High-Frequency Trading is an engineering discipline first. A strategy that is *correct but slow* loses money. The profit window for many HFT strategies is measured in microseconds: the edge disappears the moment another firm quotes a better price. A firm that can react to a market event 10 μs faster than its competitors will systematically capture fills at better prices.

This creates a direct economic incentive to reduce *every nanosecond* of latency in the signal processing chain:



2.2 The p99 Tail Latency Problem

CPU-based systems do not have a single latency — they have a *distribution*. The mean latency might be 2 μs, but the 99th percentile (p99) can be 100× higher due to:

- OS context switches and scheduler jitter
- CPU cache misses (L3 miss: ~40 ns; DRAM miss: ~100 ns)
- Hyper-threading contention
- Branch misprediction pipeline flushes
- Garbage collection (JVM/Python-based systems)

An FPGA has **no operating system, no cache hierarchy, no shared bus**. Every message takes *exactly* N clock cycles. This is the key insight.

3 NASDAQ TotalView-ITCH 5.0 Protocol

3.1 What Is ITCH?

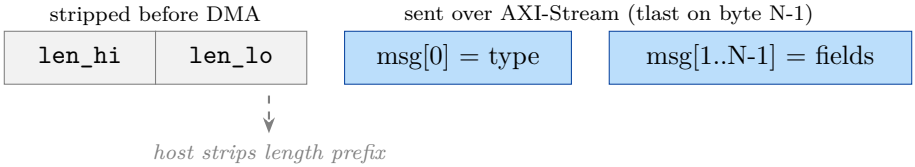
NASDAQ TotalView-ITCH is a one-directional, UDP-multicast market data feed published by NASDAQ. It carries the full limit order book for all NASDAQ-listed equities in real time:

- Every new order added to the book (`Add Order A`)
- Every order partially or fully cancelled (`Order Cancel X`, `Order Delete D`)
- Every order execution (`Order Executed E`)
- Trade reports, cross trades, halt messages, and more

On a busy trading day, NASDAQ publishes ~50 million messages. Peak rates exceed 10 million messages/second. A CPU parsing these in software with ~200 ns per message is already at 50% utilisation just to keep up.

3.2 Message Framing

ITCH messages arrive in a **SoupBinTCP** or **MoldUDP64** frame. Each message is prefixed with a 2-byte big-endian length field:



The host software (`carve_itch_slice.py`) strips the 2-byte length prefix and feeds raw message bytes to the FPGA via AXI-Stream DMA. The parser sees only raw message bytes; `tlast` asserts on the final byte.

3.3 Message Types Implemented

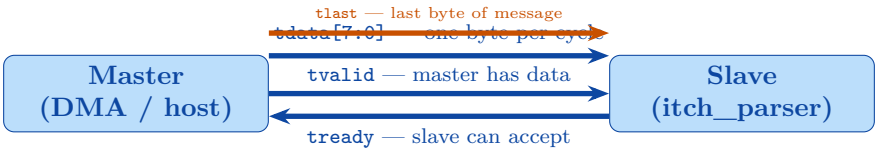
Eight ITCH 5.0 message types are decoded. Add-with-MPID (F) and Execute-with-Price (C) reuse the Add/Execute field layouts; Replace (U) carries two order references; Trade (P) is an informational print (no book change).

Type	ASCII	Name	Size	Key fields parsed
`0x41`	A	Add Order	36B	order_ref, side, shares, price
`0x46`	F	Add Order with MPID	40B	as A (+ attribution, ignored)
`0x58`	X	Order Cancel	23B	order_ref, cancelled_shares
`0x44`	D	Order Delete	19B	order_ref
`0x45`	E	Order Executed	31B	order_ref, executed_shares
`0x43`	C	Order Executed w/ Price	36B	order_ref, shares, exec price
`0x55`	U	Order Replace	35B	orig_ref, new_ref, shares, price
`0x50`	P	Trade (non-cross)	44B	order_ref, side, shares, price

4 The AXI-Stream Interface

AXI4-Stream (AXIS) is the ARM AMBA standard for streaming data between IP blocks on an FPGA. It is used throughout this design as the "plumbing" between the ITCH parser and the rest of the pipeline.

4.1 Signal Handshake



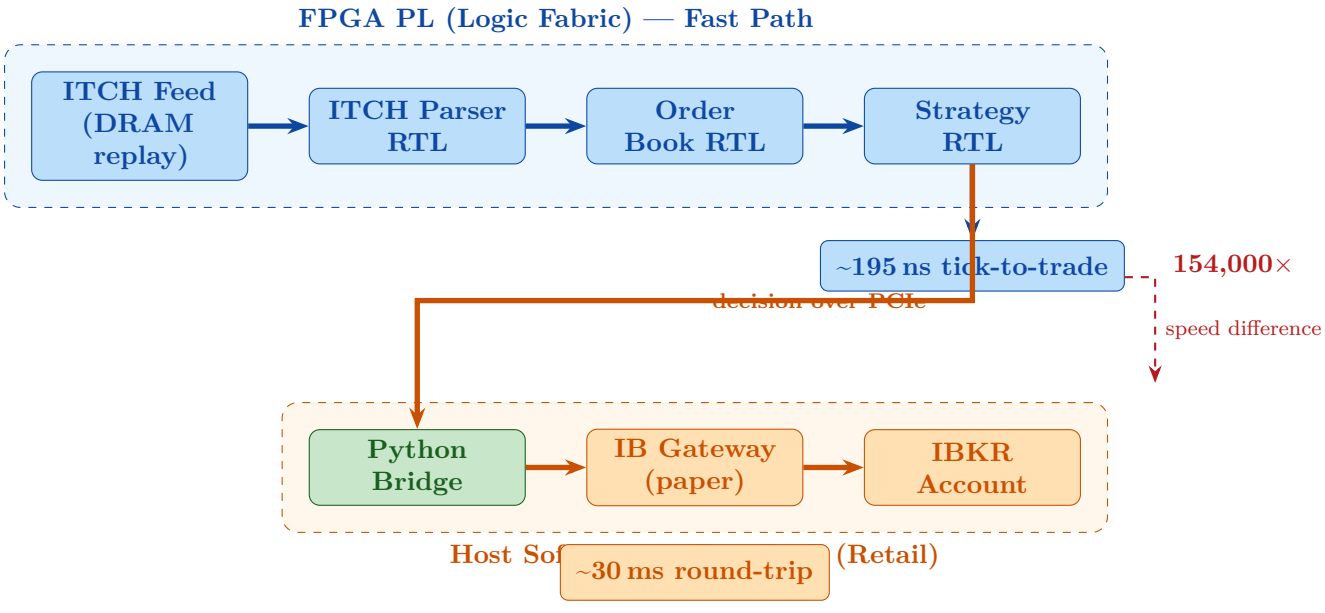
Signal	Meaning
`tvalid`	Master asserts: "I have valid data this cycle"
`tready`	Slave asserts: "I can accept data this cycle"
`tdata`	8-bit data byte (one byte per clock in this design)
`tlast`	Asserted on the *last* byte of a message
Transfer occurs when **both** `tvalid` and `tready` are high	

In this project, the parser always holds `tready = 1` (it can always accept a byte). This simplifies the interface: a transfer happens every cycle that `tvalid` is high. The parser counts bytes using `byte_cnt` and asserts `m_valid` for one cycle when the last byte (`tlast`) arrives.

5 The Two-Path Architecture

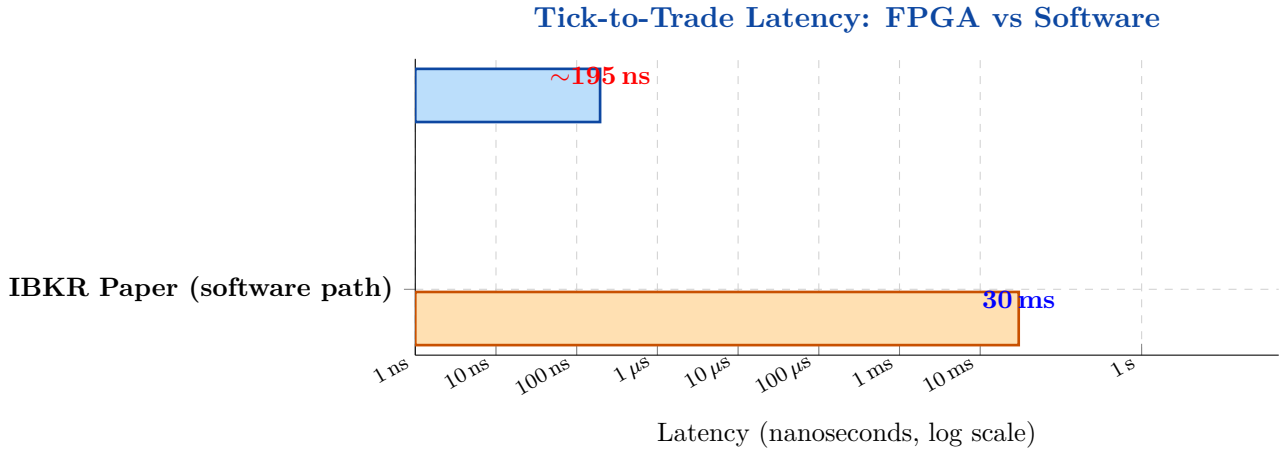
5.1 Design Decision: Honest Two-Path Split

Interactive Brokers is a retail brokerage — its API is rate-limited to ~50 messages/second and order round-trips take ~30 ms. Claiming "nanosecond trading through IBKR" would immediately destroy credibility with any HFT recruiter. The design therefore labels the two paths honestly:



5.2 The Headline Comparison

The **single most compelling artifact** of the project is this latency chart. The log scale is necessary because the two values differ by six orders of magnitude.



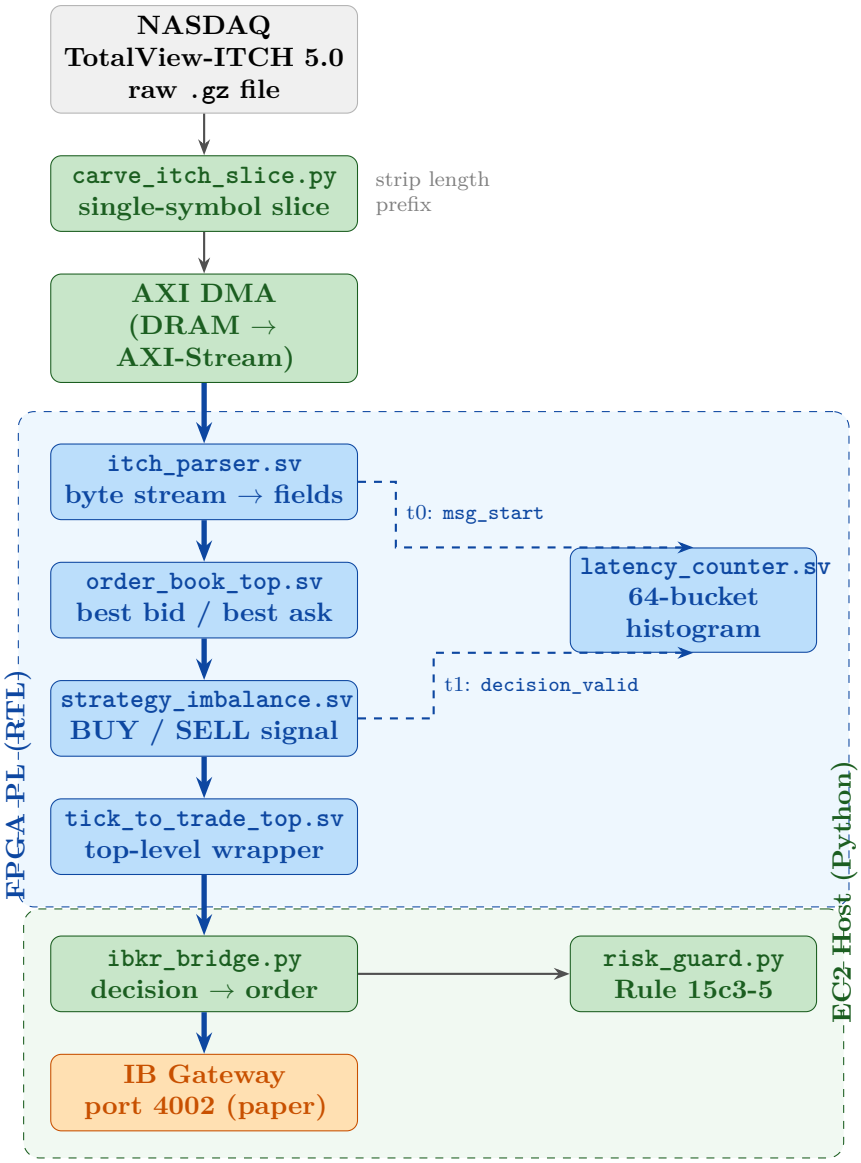
6 Design Decisions

Every key decision is documented with its rationale so interviewers can probe it.

Decision	Choice Made	Reason
Logic style	RTL (SystemVerilog) — not HLS	HFT JDs explicitly ask for RTL; interviewers write RTL on whiteboards; HLS output is opaque for debugging
Arithmetic	Fixed-point integers, no IEEE 754 floats	ITCH prices are already integers (×10,000); no rounding error; one DSP block per multiply
Delivery	Phased — M1 in weeks, then deepen	A working artefact to post sooner; reduces risk of a months-long project with nothing to show
Execution path	IBKR paper account	Real confirmations, zero financial risk; clearly labelled as the slow path
Hardware platform	AWS F1 (UltraScale+ VU9P)	PL-connected 25G Ethernet; no board purchase; real bitstream on real silicon
Simulator	QuestaSim 2021.2 (Premium)	Industry standard; supports SystemVerilog, SVA, functional coverage; already installed
Testbench	cocotb (Python)	Reuse the same Python golden model for both simulation driving and reference comparison

7 Full System Architecture

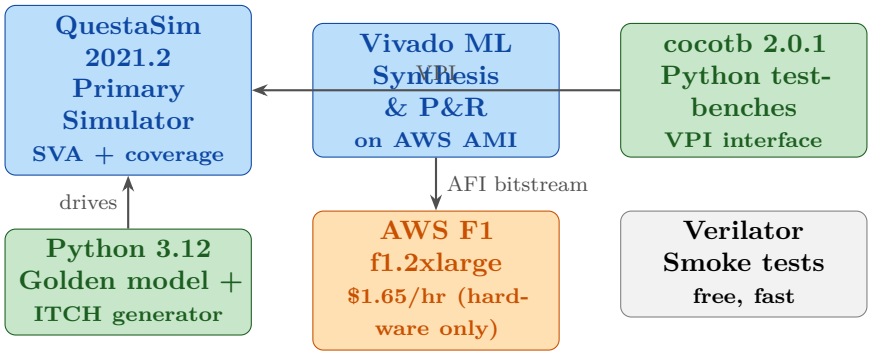
The diagram below shows every component and how data flows from the raw ITCH file to an order in a paper account.



8 Repository Layout

Path	Contents
`rtl/`	All synthesisable SystemVerilog modules
`itch_parser.sv`	AXI-Stream slave, ITCH byte decoder, field shift registers
`order_book_top.sv`	M1: 256-entry direct-mapped order book (register file)
`order_book_m2.sv`	M2: multi-level book (top 4/side), RESCAN FSM, `msg_ready` backpressure
`strategy_imbalance.sv`	Depth-weighted imbalance, spread guard, cooldown, scaled lots
`latency_counter.sv`	Free-running counter + 64-bucket histogram, AXI-Lite readout
`tick_to_trade_top.sv`	Top wrapper: wires all RTL modules together
`sim/`	Simulation infrastructure
`tb_itch_parser.py`	cocotb testbench, 6 tests (all PASS)
`Makefile`	Questa + cocotb integration
`synth_itch.py`	Synthetic ITCH message generator (no download needed)
`golden/itch_parser.py`	Python reference decoder
`golden/order_book.py`	Python reference order book
`host/`	EC2 host software
`ibkr_bridge.py`	Routes FPGA decisions to IBKR via `ib_async`
`risk_guard.py`	Pre-trade risk: size, fat-finger, rate, position, kill-switch
`data/`	Market data tooling
`carve_itch_slice.py`	Extracts single-symbol slice from ITCH `.gz`
`docs/`	This documentation (LaTeX + TikZ → PDF)

9 Toolchain and Cost



Activity	Where	Cost
RTL simulation (all iterations)	Local Ubuntu VM	Free
Synthesis sanity check	AWS EC2 (FPGA Dev AMI)	~$0.10/hr
AFI bitstream build (one-time)	AWS EC2 (Vivado)	~$2–3 total
Hardware test on real VU9P	AWS F1 f1.2xlarge	$1.65/hr

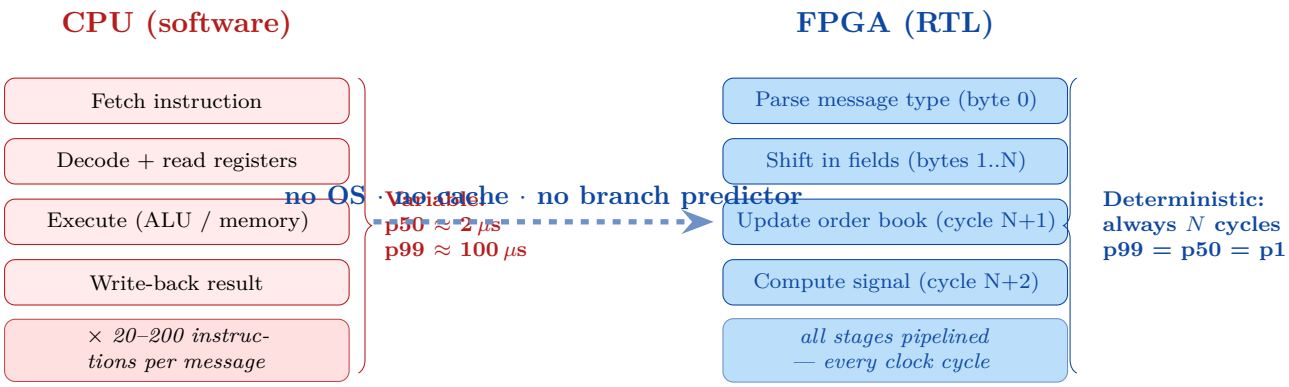
No local Vivado install required — the AWS FPGA Developer AMI includes Vivado 2022.2 pre-installed. This saves ~25 GB of disk space.

10 Why FPGAs Beat CPUs for This Task

10.1 The Pipeline Model vs the Von Neumann Model

A CPU processes one instruction at a time through a shared pipeline. Multiple operations on the same data (parse → update → compare) happen sequentially; the CPU must fetch, decode, and execute each step.

An FPGA implements all operations as **concurrent hardware**, all running simultaneously on every clock cycle:



10.2 Fixed-Point Arithmetic — No Division Required

ITCH prices are 32-bit integers representing price × 10,000 (e.g., \$150.01 is stored as `1500100`). Division is **avoided entirely** via the cross-multiplication identity:

$$\frac{a}{b} > k \quad \Longleftrightarrow \quad a \cdot k_d > k \cdot b$$

where $k = \frac{k_n}{k_d}$. For the imbalance signal with threshold $k = 1.5 = \frac{15}{10}$:

$$\text{bid\_size} \times 10 > 15 \times \text{ask\_size}$$

Both sides require only one DSP multiply block each. The comparison is a single combinational comparator. **No division, no floating-point unit, no multi-cycle latency.**

10.3 Determinism and FPGA Clock Discipline

Every flip-flop in the FPGA changes state on exactly the same rising clock edge. The critical path — the longest combinational path between two registers — determines the maximum frequency. Vivado's timing engine reports the *Worst Negative Slack* (WNS). A positive WNS means timing is met; negative means the design must be pipelined further or the clock slowed down.

At 200 MHz (5 ns period), and with a critical path of ~4 ns in the strategy module, WNS ≈ +1 ns. This leaves margin for process, voltage, and temperature variation.